

Fase 2 – Projeto 3

SmartCash: Seu Dashboard Financeiro Inteligente

O desafio desta fase

Até aqui, você trabalhou a estrutura e a apresentação. Agora, o jogo muda.

Neste projeto, você vai construir uma **aplicação web interativa**, capaz de reagir às ações do usuário, processar dados em tempo real e manter informações salvas mesmo depois que a página for fechada. É aqui que você começa a deixar de ser apenas alguém que monta interfaces para se posicionar como alguém que **programa comportamentos**, **resolve problemas com lógica** e cria experiências que realmente funcionam.

O **SmartCash** marca essa virada. Seu papel será desenvolver uma ferramenta de controle financeiro pessoal que permita registrar entradas e saídas, calcular saldo automaticamente, exibir o histórico das transações e salvar tudo localmente com **LocalStorage**.

Este projeto existe para provar que você já consegue lidar com o coração da programação front-end: **JavaScript Vanilla**, **manipulação de DOM**, **lógica de cálculo** e **persistência local de dados**.

Posicionamento do projeto

O SmartCash é um projeto de nível **intermediário**, com foco em **lógica dinâmica**, duração sugerida de **8 semanas** e execução **estritamente individual**.

Dentro da progressão da formação, ele ocupa um lugar central na **Fase 2**. Seu objetivo é consolidar os fundamentos de algoritmos e fluxo de dados antes que você avance para cenários com servidores, APIs e bancos de dados mais complexos.

Aqui, não basta “fazer funcionar”. Você precisa escrever um código que seja **organizado**, **legível**, **capaz de evoluir** e que responda em tempo real ao que o usuário faz na tela.

O cenário que justifica a aplicação

O controle financeiro pessoal é uma das dores mais comuns da vida moderna. Muita gente até tenta se organizar, mas abandona a tarefa quando encontra ferramentas burocráticas, lentas ou difíceis de usar.

Planilhas podem ser frias demais. Aplicativos completos, às vezes, exigem cadastro excessivo, conexão constante com a internet ou processos que afastam justamente quem mais precisa de simplicidade.

É nesse ponto que o SmartCash ganha relevância.

A proposta é criar uma solução **leve**, que funcione direto no navegador, preserve a **privacidade do usuário** ao salvar os dados localmente e entregue uma resposta imediata: registrar uma despesa ou uma receita e ver o saldo atualizado na mesma hora.

Quem está envolvido nesse desafio

O projeto simula uma necessidade real de mercado.

O **cliente** é uma startup fictícia de educação financeira que quer validar um protótipo de uma calculadora de gastos simplificada.

O **usuário final** são pessoas que precisam de uma maneira simples de anotar gastos diários e enxergar, de forma clara, para onde o dinheiro está indo — por exemplo, em categorias como alimentação, lazer e transporte.

Você, como **desenvolvedor**, precisa demonstrar que consegue controlar **fluxos de dados**, realizar **cálculos matemáticos**, atualizar a interface sem recarregar a página e manter a experiência consistente do início ao fim.

Objetivo do projeto

Ao final das 8 semanas, você deverá entregar uma **SPA (Single Page Application)** funcional, desenvolvida com **HTML5**, **CSS3** e **JavaScript Vanilla**, capaz de realizar o gerenciamento completo de entradas e saídas financeiras.

O foco sai do visual puramente estático e entra no campo da aplicação viva: uma interface que responde, calcula, armazena e apresenta dados de forma útil e imediata.

A visão de sucesso é clara: o usuário deve conseguir usar a ferramenta para controlar um orçamento real, com uma interface que reage instantaneamente aos comandos, sem necessidade de refresh e sem perder os dados ao fechar o navegador.

O que sua aplicação precisa entregar

Sua entrega principal será uma aplicação funcional de gestão financeira pessoal.

Ela deverá incluir as seguintes funcionalidades:

- **Registro de transações**, com formulário para inserção de descrição, valor e tipo (**Entrada** ou **Saída**)
- **Painel de resumo em cards**, exibindo dinamicamente o total de entradas, o total de saídas e o saldo total, com atualização automática a cada interação
- **Lista de extrato**, mostrando as transações em ordem cronológica e oferecendo opção de exclusão individual

- **Persistência de dados com LocalStorage**, garantindo que os dados permaneçam salvos mesmo após fechar o navegador ou recarregar a página
- **Feedback visual de status**, com diferenciação clara, por meio de cores e ícones, entre valores positivos e negativos

O que não faz parte do escopo

Nesta fase, o objetivo não é trabalhar com frameworks ou banco de dados externos.

Por isso, **não deve ser utilizado**:

- React
- Angular
- qualquer outro framework
- MySQL
- MongoDB
- APIs de terceiros
- bancos externos como Firebase ou SQL

O centro do projeto é a **lógica pura**, a **manipulação do DOM** e o uso de recursos nativos do navegador.

Como o projeto será construído ao longo das 8 semanas

O desenvolvimento do SmartCash está dividido em quatro etapas quinzenais. A evolução começa pela interface, passa pela lógica e culmina na persistência e no refinamento da experiência.

Marco 1: Prototipagem e Estrutura Funcional

Semanas 1 e 2

Neste primeiro momento, você vai construir o layout completo da aplicação com **HTML** e **CSS**.

O entregável desta etapa é a interface contendo todos os elementos necessários: **formulário de entrada**, **cards de resumo** e **área do extrato**.

O critério para avançar é que a interface esteja pronta para receber comportamento dinâmico, com **IDs** e **classes** bem definidos para futura manipulação via JavaScript.

Marco 2: O Cérebro da Aplicação

Semanas 3 e 4

Agora, você começa a ativar a inteligência do sistema.

O entregável desta fase é a implementação das funções básicas de captura de eventos, como cliques, e da lógica de cálculo: soma das entradas, soma das saídas e cálculo do saldo.

O critério de transição é direto: ao preencher o formulário, os valores devem aparecer no **console do navegador** e os cards precisam ser atualizados na tela, ainda que os dados desapareçam ao atualizar a página.

Marco 3: Persistência e Memória

Semanas 5 e 6

Nesta etapa, a aplicação deixa de ser temporária.

O entregável será a integração com o **LocalStorage**, permitindo salvar transações no navegador e recuperá-las automaticamente ao abrir a página novamente.

O critério para passar de fase é que o usuário consiga fechar o navegador, voltar depois e encontrar seus dados financeiros ainda visíveis e corretos.

Marco 4: Refinamento, Validação e Deploy

Semanas 7 e 8

Com a aplicação funcional e persistente, chega o momento de elevar a qualidade.

O entregável final inclui:

- validações de formulário
- tratamento de erros
- polimento visual com animações simples de entrada e saída
- publicação da aplicação com link funcional

O checkpoint final desta fase exige atenção a dois pontos centrais: **bugs de arredondamento monetário** e **organização do código segundo princípios de Clean Code**.

Conceitos fundamentais que você precisa dominar

Para dar vida ao SmartCash, você precisa sair da lógica de marcação e entrar no fluxo da programação.

Lógica de programação e algoritmos

Você deverá saber decompor o cálculo de um saldo financeiro em etapas claras, utilizando estruturas de decisão como **if/else** e estruturas de repetição.

Manipulação de DOM

Será necessário aprender a **selecionar, criar e alterar elementos HTML dinamicamente** com JavaScript, para que a tela responda às ações do usuário em tempo real.

Estruturas de dados: arrays e objetos

As transações deverão ser organizadas em uma **lista de objetos**, facilitando armazenamento, leitura, cálculo e busca de informações.

Métodos de array de alta ordem

Você precisará dominar métodos como **.map()**, **.filter()** e principalmente **.reduce()**, já que eles são essenciais para somar entradas, saídas e gerar resumos com mais clareza e performance.

Persistência com LocalStorage

Você deverá entender como funciona a **Web Storage API**, salvando e recuperando dados em formato **JSON** diretamente no navegador.

Gestão de eventos

Capturar cliques, envios de formulário e outras interações será parte central da aplicação. Cada ação do usuário deve disparar a lógica certa no momento certo.

Fundamentos de Clean Code

À medida que o projeto cresce, a clareza do código se torna obrigatória. Por isso, você precisará aplicar boas práticas de nomenclatura de variáveis e funções, mantendo o sistema compreensível e sustentável.

Estratégia de estudo com IA

Neste projeto, a IA pode funcionar como uma parceira de **pair programming**, especialmente no momento de estruturar a lógica e revisar decisões de implementação.

A recomendação é clara: dedique **70% do seu tempo ao JavaScript**.

Comece aprendendo como capturar valores de um formulário. Em seguida, foque em como armazená-los em um **array de objetos**. A parte visual mais refinada pode ficar para momentos de pausa mental, quando você já tiver avançado na inteligência da aplicação.

Alguns prompts úteis para esse processo são:

- “Estou desenvolvendo um app financeiro. Explique-me, passo a passo, a lógica para percorrer uma lista de transações e separar o que é entrada do que é saída, somando os totais de cada uma.”

- “Atue como um revisor de código sênior. Analise minha função de ‘excluir transação’ [colar código] e me diga se existe uma forma mais performática ou limpa de remover esse item do meu Array e atualizar a tela.”
- “Como eu transformo um Array de Objetos em uma string JSON para salvar no LocalStorage e como faço o processo inverso ao carregar a página?”

Você também pode usar a IA de acordo com cada milestone:

- Para o **Marco 1**, peça explicações sobre **querySelector** e **addEventListener**
- Para o **Marco 2**, concentre-se em **.push()**, **.forEach()** e **.reduce()**
- Para o **Marco 3**, aprofunde-se em **JSON.stringify()** e **JSON.parse()**

Para treinar depuração, use este protocolo:

“Crie um erro proposital em uma função de cálculo de saldo e me peça para encontrar o bug. Quero treinar meu poder de depuração.”

O que você deve aprender ao final deste projeto

Ao concluir o SmartCash, você deverá ser capaz de:

- dominar a **manipulação do DOM**, inserindo, removendo e atualizando elementos da interface com base nas ações do usuário
- aplicar **lógica de programação** para resolver problemas de agregação de dados, como cálculo de saldo e filtragem de transações
- implementar um fluxo básico de **CRUD**, permitindo criar transações, ler dados salvos e deletar registros
- executar a **persistência de dados no navegador**, mantendo o estado da aplicação entre diferentes sessões
- estruturar o código JavaScript de maneira **modular**, organizada e sem repetição desnecessária de lógica, seguindo o princípio **DRY**
- validar dados de entrada para impedir erros de execução e garantir processamento confiável
- desenvolver habilidades de **debugging**, utilizando o console do navegador para encontrar e corrigir falhas de lógica e erros de referência

Requisitos e restrições

O SmartCash possui regras de negócio e limitações técnicas que precisam ser respeitadas para que o aprendizado aconteça da forma certa.

Requisitos funcionais

O sistema deve:

- permitir que o usuário cadastre uma transação com **nome e valor**
- tratar **valores positivos como receitas** e **valores negativos como despesas**

- somar automaticamente todas as entradas, todas as saídas e exibir o **saldo líquido em tempo real**
- listar as transações de forma organizada e permitir que qualquer item seja apagado por meio de um botão de exclusão
- utilizar **obrigatoriamente o LocalStorage**, para que os dados não sejam perdidos ao fechar ou atualizar a página

Requisitos não funcionais

- **Validação de formulário:** o envio deve ser bloqueado se os campos estiverem vazios ou se o valor não for numérico válido
- **Lógica de dados:** as informações devem ser gerenciadas em um **array de objetos** dentro do JavaScript
- **Responsividade:** a interface deve ser adaptada para celulares, garantindo leitura adequada do extrato em telas menores

Restrições técnicas

Estas são as regras do jogo nesta fase:

- **JavaScript Vanilla obrigatório:** é proibido o uso de bibliotecas ou frameworks como React, Vue, jQuery e similares
- **Sem banco de dados externo:** não é permitido usar APIs de terceiros, Firebase, SQL ou qualquer estrutura de armazenamento externa
- **Organização de arquivos obrigatória:** o projeto deve estar separado em `index.html`, `style.css` e `script.js`

Recursos e documentação para sustentar a construção

Como este é um projeto centrado em JavaScript, você precisará usar ferramentas de diagnóstico e fontes confiáveis para entender o comportamento da aplicação.

Ferramental de execução e debug

- **Console e DevTools do navegador:** uso intensivo da aba **Console** e da aba **Application**, especialmente para monitorar o LocalStorage
- **Debugger do VS Code:** configuração de breakpoints para acompanhar a execução do código linha a linha

Documentação de referência

- **MDN JavaScript Guide:** referência para sintaxe de objetos, arrays e manipulação de eventos
- **JSON Methods (MDN):** documentação de apoio para uso de `JSON.stringify` e `JSON.parse`

Curadoria de conteúdo

- **Guias visuais de métodos de array:** materiais práticos para entender como transformar e agregar dados
- **Web Storage API:** explicações técnicas sobre como o navegador armazena dados localmente sem expiração

Riscos e pontos de atenção

Nesta fase, os riscos deixam de ser principalmente visuais e passam a ser lógicos e funcionais.

Gargalos de lógica

Um dos erros mais comuns está na **tipagem de dados**. Se o JavaScript interpretar valores do formulário como texto, em vez de número, os cálculos podem falhar. É o caso clássico em que `10 + 10` vira `1010`.

Outro risco está em **referência de variáveis**, quando você tenta acessar uma transação fora do escopo em que ela foi criada.

Riscos de persistência

Um dos problemas mais delicados é o **JSON corrompido**: salvar ou ler dados do LocalStorage sem converter corretamente entre string e objeto pode impedir a aplicação de carregar como deveria.

Impacto no desempenho

Outro ponto de atenção está na **renderização excessiva**. Recriar toda a lista de transações a cada pequena alteração pode comprometer a experiência se houver muitos dados.

Plano de contingência

Se a tela parar de responder, sua primeira ação deve ser abrir o **console do navegador** e procurar erros em vermelho.

Se a funcionalidade de exclusão começar a falhar, teste após adicionar vários itens, verificando se o índice dos dados no JavaScript ainda corresponde ao que o usuário vê na tela.

Critérios de aceite

O projeto só pode ser considerado concluído quando todos os itens abaixo estiverem validados.

Checklist funcional

- O usuário consegue adicionar transações com nome e valor, e elas aparecem instantaneamente na lista
- Os cards de resumo (**Entradas**, **Saídas** e **Saldo Total**) exibem os cálculos corretos após cada operação
- A funcionalidade de exclusão remove o item da tela e atualiza os totais automaticamente
- Os dados são recuperados do **LocalStorage** sempre que a página é carregada ou atualizada
- O formulário bloqueia entradas inválidas, como campos vazios ou valores não numéricos

Qualidade do código e engenharia

- O projeto usa apenas **JavaScript Vanilla**, sem frameworks ou bibliotecas externas
- A atualização da interface é feita por **manipulação de DOM**, sem recarregar a página inteira
- Todos os valores numéricos são convertidos de **string para number** antes dos cálculos
- As funções possuem responsabilidades únicas e as variáveis têm nomes claros e descritivos

Experiência do usuário

- O layout é responsivo e funciona sem quebras em telas de celular
- Existe distinção visual clara entre **receitas** e **despesas**

Condições de entrega

- O código-fonte está disponível em **repositório público no GitHub**
- O deploy foi realizado, por exemplo via **GitHub Pages**, com link funcional testado e acessível a qualquer usuário

Possibilidades de expansão

Depois de cumprir o escopo obrigatório, você poderá transformar o SmartCash em uma aplicação mais robusta e analítica.

Visualização de dados

Adicionar **gráficos de pizza ou barras** com bibliotecas como **Chart.js** ou **Google Charts** para representar visualmente os gastos por categoria.

Filtros e pesquisa

Criar uma barra de busca para localizar transações específicas ou filtros que permitam ver apenas entradas e saídas de um mês específico.

Categorização avançada

Permitir que o usuário defina categorias próprias, como **Aluguel**, **Freelance** ou **Pet**, com ícones personalizados.

Exportação de dados

Criar a opção de exportar o extrato em **CSV** ou **PDF**, permitindo uso posterior em outras ferramentas como Excel.

Sistema de metas

Adicionar uma meta de economia mensal e uma barra de progresso que mostre o quanto ainda falta para atingi-la.

Feedback sonoro e microinterações

Incluir sons sutis ao confirmar uma transação ou animações comemorativas, como confete ao atingir uma meta.

Preparação para o backend

Estruturar o código de forma que facilite a futura troca do **LocalStorage** por uma **API real**, introduzindo conceitos como **async/await** e **fetch**.

Fechamento

O SmartCash é o projeto em que a interface deixa de ser apenas aparência e passa a se comportar como sistema.

Ao final dele, você não terá apenas uma tela bonita: terá uma aplicação que escuta o usuário, processa informações, atualiza a interface em tempo real e preserva dados entre sessões.

É aqui que sua base em front-end ganha profundidade. É aqui que você começa a provar que sabe programar lógica de verdade.